Pureflow Architecture Guide

Contents

Pureflow Architecture Guide . . . 2
- Audience . . . 2
- Scope . . . 2
- Recommended Reading Order . . . 3
- Why This Guide Exists . . . 3
- Relationship to Other Docs . . . 3

System Overview . . . 3
- Pureflow Overview . . . 3
- Original FBP Roots . . . 4
- Key Terms . . . 4
- Why This Architecture Matters . . . 5
- Layered Architecture . . . 5
- Crate Responsibilities . . . 6
- Runtime Data Flow . . . 7
- Why WASM Helps Here . . . 7
- Why `asupersync` Instead of Tokio . . . 8
- Metadata Surfaces and Usage . . . 9
  - Metadata JSONL record families . . . 9
  - Run summary JSON fields . . . 9
- Important Examples . . . 10
  - CLI usage . . . 10
  - Workflow document shape . . . 10
  - Contract/capability alignment example (Rust) . . . 10
- Boundary Decisions . . . 11
- Open Questions . . . 11
- Chapter Takeaways . . . 12

Runtime Model . . . 12
- Intent . . . 12
- Execution Lifecycle . . . 13
  - Runtime Layers at a Glance . . . 14
- Runtime Phases and Boundaries . . . 14
  - Phase 1: Definition and Safety Checks . . . 15
  - Phase 2: Execution and Message Transport . . . 15

Phase 3: Terminalization and Summary . . . 16
Error Handling Model . . . 16
Observability Touchpoints . . . 16
Operational Notes . . . 17
Runtime Takeaways . . . 18
Package and Contract Boundaries . . . 18
Intent . . . 18
Crate Map by Responsibility . . . 18
Public Contract Surfaces . . . 19
Invariants . . . 20
Compatibility and Versioning Expectations . . . 20
Workflow Format Compatibility . . . 20
Contract/Capability Compatibility . . . 20
Boundary Diagram . . . 21
Review Checklist . . . 21
Boundary Takeaways . . . 21
Diagrams and Flows . . . 22
Intent . . . 22
Diagram Inventory . . . 22
Source of truth . . . 22
Rendering Notes (Quarto + Mermaid) . . . 22
Frankenmermaid Conventions . . . 23
Canonical Flows to Preserve . . . 23
Suggested Next Diagrams . . . 23
Review Workflow . . . 24
Diagram Takeaways . . . 24

Pureflow Architecture Guide

Audience

This guide is for engineers who need a system-level understanding of Pureflow.

It is written for humans first. The point is not to compress the design into a set of symbols for tooling; the point is to explain why the system is shaped the way it is, what each layer is protecting, and what tradeoffs are worth remembering when the code changes.

Scope

- High-level architecture and subsystem responsibilities.
- Runtime flow and execution model.
- Package and contract boundaries.

- Diagram-backed explanations for key pathways.

Recommended Reading Order

If you are new to the codebase, read this guide in order:

1. System Overview for the big picture and the main design choices.
2. Runtime Model for how work actually moves through the engine.
3. Package and Contract Boundaries for why the crates are split the way they are.
4. Diagrams and Flows for the conventions behind the figures.

Why This Guide Exists

Pureflow has enough moving parts that "what the code does" is not the same as "why the code is arranged this way." This guide is here to bridge that gap for the next person who opens the repo with coffee in hand and wants the short version before they start reading Rust.

Relationship to Other Docs

- API reference is generated separately via `cargo doc`.
- Retrieval-oriented facts live in `docs/llm/`.

System Overview

Pureflow Overview

Pureflow is an experimental Flow-Based Programming engine in Rust. It validates workflow documents, executes node graphs through bounded channels, and emits machine-facing metadata and run summaries.

The system is built to keep three things visible at once: what the workflow looks like, how the run is executing, and where the boundaries of responsibility actually sit. That is what makes the rest of the guide worth reading slowly.

If you want a mental model, think of Pureflow like a shipping hub:

- the workflow document is the route map,
- nodes are processing stations,
- edges are conveyor belts with finite space,
- metadata is the camera footage and scanner log.

At a high level, Pureflow separates:

- static graph and format validation,
- runtime execution and orchestration,
- contract/capability policy,

- introspection and CLI surfaces.

If you only remember one thing from this chapter, remember that those concerns are intentionally not mixed together. Most of the design choices in the rest of the guide are there to protect that separation.

Original FBP Roots

Pureflow is based on J. Paul Morrison's original Flow-Based Programming model. That model matters here because it gives the architecture a very practical shape:

- components behave like long-lived processes instead of one-shot functions,
- data moves over bounded channels instead of through hidden shared state,
- execution is streaming and demand-driven instead of batch-only,
- backpressure becomes visible when downstream work cannot keep up,
- the graph stays loosely coupled because each node only knows its ports.

That fit is deliberate because it keeps the system honest. You can see where data is waiting, where work is blocked, and where a contract mismatch is really a wiring problem rather than a mysterious runtime bug.

If you are coming from a more traditional scheduler or queue-heavy design, this is the part to notice: FBP is not trying to hide the graph. It is trying to make the graph the thing you reason about.

Key Terms

The guide uses a few runtime terms in a precise way:

- bounded channels: the edges between nodes have fixed capacity, so sends can block or fail instead of growing without limit.
- backpressure: the signal that an upstream node is producing faster than the downstream side can consume.
- node: one unit of work in the workflow graph.
- port: a named input or output on a node.
- packet: one item of data moving across a port.
- metadata: machine-readable records that describe what happened during a run.
- capability: a declared permission or restriction on what a node may do.
- contract: the expected shape and behavior of a node at the workflow boundary.

You do not need to memorize the glossary on first read. The point is to keep the language stable enough that a junior engineer can read one chapter and the next without translating the vocabulary in their head.

Why This Architecture Matters

The architecture is built to make the important things visible instead of leaving them implicit.

- Workflow shape is validated before execution, so a bad document fails early and clearly.
- Bounded channels make pressure a first-class event instead of a hidden scheduler accident.
- Runtime metadata records what happened while the run is still live, which makes debugging and replay easier than reconstructing a story from logs after the fact.
- Capability and contract checks keep the node boundary honest, which matters more as execution modes get more diverse.
- The same design gives Pureflow room to run native nodes, WASM nodes, and future adapters without rewriting the whole graph model.

That combination is what makes the system interesting: it behaves like a serious execution engine, but it still tries to keep the logic comprehensible to a person reading the code for the first time.

Pureflow is trying to be the kind of system you can debug by reading from top to bottom, not the kind you need to reverse-engineer from a pile of moving parts.

Layered Architecture

The figure below shows the major crates and handoffs.

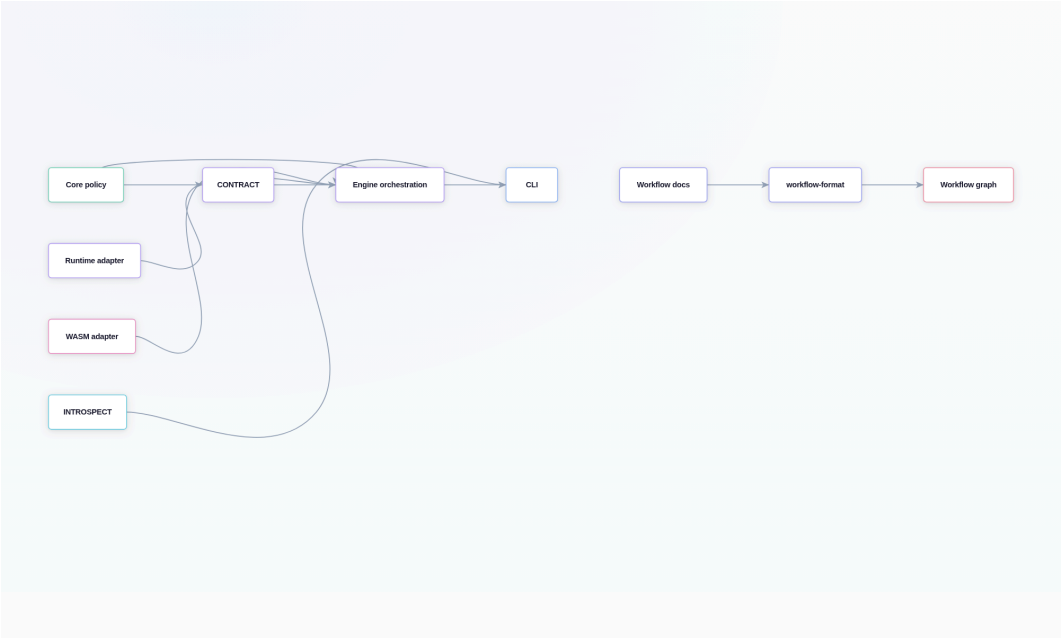

Figure 1: Layered architecture diagram

Crate Responsibilities

Crate	Primary responsibility	Used by
pureflow-types	Validated identifier primitives (WorkflowId, NodeId, PortId)	all crates
pureflow-workflow-format	External format decode + versioning (pureflow_version)	CLI + loaders
pureflow-workflow	Static topology model + structural validation	contract/introspection/engine
pureflow-core	Runtime-facing traits, ports, metadata, errors, capabilities	runtime/engine/cli/wasm
pureflow-contract	Node contract model + validation against workflow/capabilities	engine/introspection/cli
pureflow-introspection	Read-only workflow+contract+capability projections	CLI inspect/explain
pureflow-runtime	asupersync runtime wrapper + node observer boundary	engine

Crate	Primary responsibility	Used by
pureflow-engine	Workflow orchestration, registry scheduling, summary/policy	CLI
pureflow-wasm	Wasmtime Component Model batch execution adapter	engine/CLI
pureflow-cli	User and automation entrypoint	humans/CI
pureflow-test-kit	Builders and helpers for tests/examples	tests/docs

Runtime Data Flow

Another way to read this: CLI is the dispatcher, engine is the floor manager, runtime is the shift supervisor, and each node is a station doing one job.

This picture matters because it matches how the code is split. The CLI handles entry and exit, the engine decides what should happen, the runtime makes the work actually run, and the nodes stay focused on their own logic.

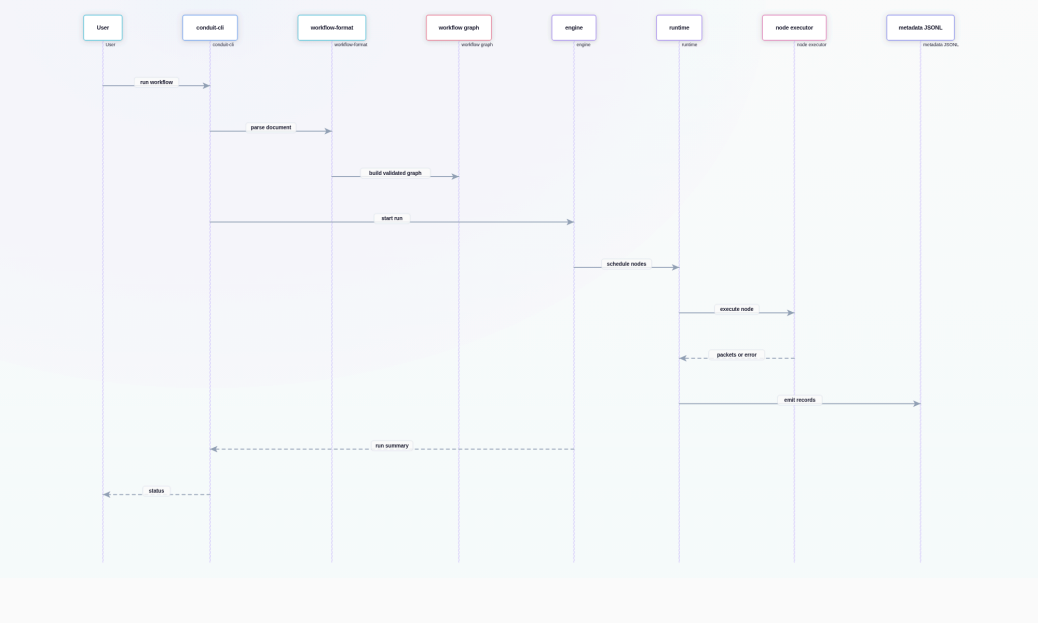

Figure 2: Runtime data flow diagram

Why WASM Helps Here

WASM is useful in Pureflow because it gives you a narrow, portable execution boundary without asking the rest of the system to trust a whole host runtime.

The value here is not just “WASM is safer.” It is that WASM gives Pureflow a second implementation path without letting that path leak into the core model.

- A WASM node is easier to ship and inspect than a native plugin with deep host reach.
- The host owns the channels, so the graph still looks like Pureflow even when a node is compiled to a different execution target.
- The capability boundary stays explicit. If a guest cannot reach the network or filesystem, that limitation is visible in the adapter instead of hidden in the node implementation.
- The batch-style component model keeps the WASM story focused on data in, data out, which is a good fit for flow graphs.

This means WASM is not just a deployment trick. It is a way to make extension safer, more portable, and easier to reason about when the engine starts to grow.

That makes it a good fit for a system that wants strong boundaries and a clear story for how untrusted or semi-trusted code fits into the graph.

Why `asupersync` Instead of Tokio

Tokio is a strong general-purpose async ecosystem, but Pureflow does not need a general-purpose app runtime to explain its own model. It needs a substrate that stays out of the way while still giving the engine task-tree execution, cancellation, and bounded channel behavior.

The goal is to keep the runtime layer boring in the right way: it should execute Pureflow’s model, not define a second one.

That is why `asupersync` sits underneath `pureflow-runtime` rather than being the public model itself.

- Pureflow owns the graph, ports, contracts, metadata, and capability story.
- `asupersync` provides the execution machinery that makes those ideas run.
- The boundary stays narrow, which makes it easier to test and easier to replace later if the project ever needs to.
- The public API does not inherit Tokio’s larger vocabulary when Pureflow only needs a smaller one for workflow execution.

There is nothing wrong with Tokio in general. The point is simply that Pureflow is trying to teach and preserve its own runtime story, not inherit a broader framework story it does not need.

That makes the boundary easier to test, easier to describe, and easier to swap later if the project ever needs to.

Metadata Surfaces and Usage

Pureflow emits two machine-facing outputs:

- metadata JSONL stream (event records),
- `pureflow run --json` terminal summary.

The short version: JSONL is the play-by-play, while the summary JSON is the box score at the end of the game.

The two outputs serve different readers. One is for tracing the run while it is happening. The other is for the final yes/no decision when the run is over.

Metadata JSONL record families

Use the first table when you are following the shape of a normal run. It is the timeline view.

record_type	Produced by	Why it matters
execution_context	runtime observer boundary	Ties records to workflow, node, attempt, and cancellation state.
lifecycle	runtime + engine	Shows node start, completion, failure, and cancellation transitions.
message	port boundary instrumentation	Shows packet movement without exposing payload bytes.
queue_pressure	port reserve/recv boundaries	Shows capacity and queue state when backpressure matters.

Use the second table when you are looking for what went wrong, or what side effects were intentionally observed.

record_type	Produced by	Why it matters
error	node/workflow failure paths	Gives a stable error taxonomy for branching, retry policy, and diagnostics.
external_effect	node integration code	Makes tool, service, database, and API side effects explicit.

Run summary JSON fields

Field	Meaning	Notes
status	terminal state (completed, failed, cancelled)	primary machine decision field

Field	Meaning	Notes
error	terminal error object or null	mirrors stable Pureflow error shape
workflow	workflow identity and shape counts	quick context for automation
metadata	output metadata path and record totals	ties summary to JSONL artifact
summary	node scheduling/completion/failure counts	execution outcome snapshot

Important Examples

CLI usage

```
# Validate workflow structure and format
cargo run -p pureflow-cli -- validate examples/native-linear-etl.workflow.json

# Inspect topology/contracts/capabilities
cargo run -p pureflow-cli -- inspect examples/native-linear-etl.workflow.json

# Run and emit machine-facing summary
cargo run -p pureflow-cli -- run --json \
  examples/native-linear-etl.workflow.json \
  /tmp/pureflow-native-linear.metadata.jsonl
```

Workflow document shape

```
{
  "pureflow_version": "1",
  "id": "native-linear-etl",
  "nodes": [
    { "id": "source", "inputs": [], "outputs": ["rows"] },
    { "id": "transform", "inputs": ["rows"], "outputs": ["cleaned"] },
    { "id": "sink", "inputs": ["cleaned"], "outputs": [] }
  ],
  "edges": [
    {
      "source": { "node": "source", "port": "rows" },
      "target": { "node": "transform", "port": "rows" },
      "capacity": 2
    }
  ]
}
```

Contract/capability alignment example (Rust)

```
use pureflow_contract::{
    Determinism, ExecutionMode, NodeContract, PortContract, SchemaRef,
    validate_workflow_contracts,
};
```

```rust
use pureflow_core::{
    RetryDisposition,
    capability::{NodeCapabilities, PortCapability, PortCapabilityDirection},
};

// Build contracts and capabilities for each workflow node, then validate:
validate_workflow_contracts(&workflow, &contracts, &capabilities)?;
```

This is a key authoring boundary for both humans and future LLM assistants: workflow topology, contracts, and capabilities must match before execution. If those three disagree, Pureflow treats it like a wiring mismatch and stops before runtime surprises can happen.

That rule is part of the architecture's value. It catches structural mistakes before they turn into runtime folklore.

Boundary Decisions

These are the boundaries worth remembering while reading the code:

Crate	Owns	Why it matters
pureflow-workflow	Structural graph honesty only.	Keeps shape validation separate from policy.
pureflow-contract	Contract/capability alignment and schema compatibility.	Catches wiring mismatches before runtime.
pureflow-core	Runtime interfaces and metadata shape.	Defines the Pureflow-owned runtime vocabulary.
pureflow-runtime	The asupersync adapter boundary.	Hides substrate details behind Pureflow types.
pureflow-wasm	Wasmtime and WIT/component ABI details.	Keeps WASM specifics out of core.

If you are skimming the code, this table is the chapter's spine. Anything that crosses one of these lines should be obvious in the types or the module name.

Open Questions

These are the points where code and docs suggest open questions or evolving policy. I recommend focused follow-up review on each.

They are not red flags so much as reminders that the design is still learning where its hard edges need to be.

Open area	Current state	Why follow-up matters
Feedback loops	Cycle support exists, but production semantics are not fully documented.	Cycles change startup, deadlock, and cancellation behavior.
Process execution mode	ExecutionMode::Process exists, but the adapter contract is future-facing.	This could become a real enforcement boundary later.
External effects	external_effect records exist, but naming guidance is loose.	Audit trails need stable vocabulary.
Queue pressure	Kinds are defined, but thresholds and interpretation rules are not.	People need guidance, not just record names.
WASM capability evolution	Import-free mode rejects declared effects today.	Host imports will need clear migration rules.

Think of this as a short list of design work, not a list of defects.

Chapter Takeaways

Pureflow works best when the workflow document, runtime behavior, and capability policy remain separate things that meet at the boundary instead of collapsing into one blurry execution blob.

That is the recurring theme in this chapter. Bounded channels, explicit contracts, and narrow adapters all serve the same goal: keep the graph legible enough that a human can reason about it without carrying the whole system in their head at once.

If the architecture continues to feel coherent as the code grows, it is because those boundaries are still doing their job.

Runtime Model

Intent

Explain how execution proceeds from workflow input through node scheduling, message flow, metadata emission, and terminal summary.

The short version is that Pureflow tries to make execution feel ordinary: validate first, run second, record what happened as it happens, and stop with

a clear terminal state. That may sound conservative, but it is what keeps the system understandable when workflows start to get interesting.

The chapter is organized the same way a run happens: first the setup, then the movement of work, then the point where the run becomes a finished artifact you can inspect.

Execution Lifecycle

Pureflow runtime behavior is coordinated by three layers:

- `pureflow-cli` handles input/output and user-facing options.
- `pureflow-engine` validates execution preconditions, builds run context, and orchestrates node execution.
- `pureflow-runtime` bridges engine scheduling into the underlying async runtime and observer hooks.

That split is not accidental. It keeps the user-facing command path, the policy layer, and the execution substrate from collapsing into one giant abstraction.

At a high level, execution follows this order:

1. Parse workflow document (`pureflow-workflow-format`).
2. Build/validate graph shape (`pureflow-workflow`).
3. Validate contracts and capabilities (`pureflow-contract` + `pureflow-core`).
4. Construct runtime registry for node executors (native and/or WASM).
5. Schedule nodes and move packets over bounded ports.
6. Emit lifecycle/queue/message/error metadata records.
7. Produce terminal `WorkflowRunSummary` for CLI output.

Read that list as a story about the run rather than a checklist. Each step narrows the question from “is this workflow valid?” to “what happened on this particular execution?”

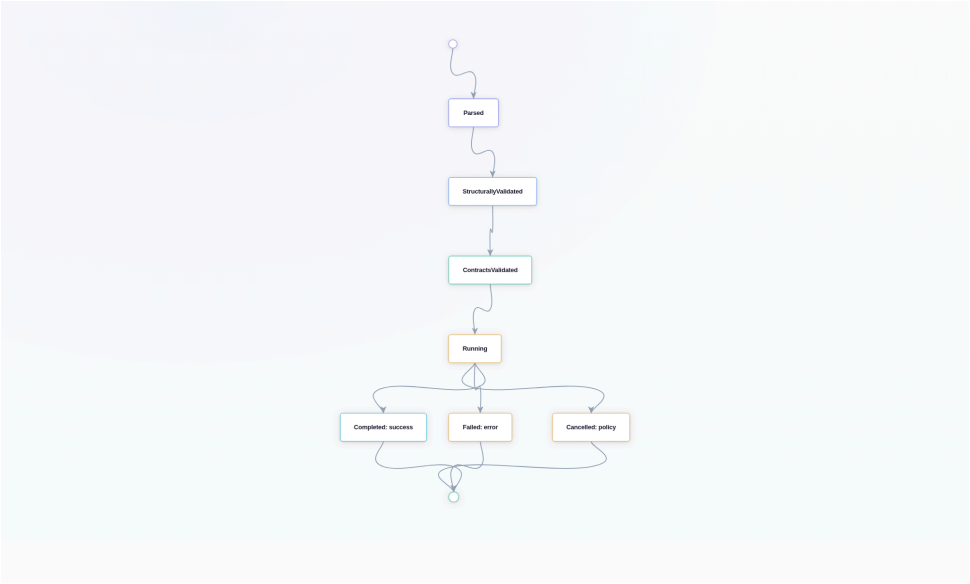

Figure 3: Runtime lifecycle state diagram

Runtime Layers at a Glance

This table is the simple map of who does what during a run.

Layer	What it does	Reader takeaway
pureflow-cli	Accepts input, prints results, and exposes commands.	This is the human and automation entrypoint.
pureflow-engine	Validates the run, builds context, and orchestrates execution.	This is where workflow policy turns into scheduling.
pureflow-runtime	Bridges engine calls into the async substrate and observers.	This is the execution seam, not the public model.

Runtime Phases and Boundaries

The phases below are the practical way to think about a run from start to finish.

The phase table below is the fast version. The sections that follow unpack each step in plain language.

Phase	What happens	What to notice
Definition and safety checks	Validate identifiers, graph shape, contracts, and schema compatibility.	Failures here are authoring issues, not node bugs.

Phase	What happens	What to notice
Execution and message transport	Move packets through bounded channels with visible pressure.	Backpressure is part of the design, not a side effect.
Terminalization and summary	Emit final records and the terminal run summary.	The stream is for forensics; the summary is for decisions.

Phase 1: Definition and Safety Checks

Before any node logic runs, Pureflow enforces static safety:

- identifiers and format version are validated,
- graph connectivity and port references are validated,
- contract/capability alignment is checked,
- schema compatibility across connected ports is checked.

This keeps runtime failures focused on execution concerns rather than document shape mistakes.

It also keeps the guide honest for junior readers: if a workflow fails before runtime starts, that is usually not a “node problem.” It is a contract, topology, or capability problem.

That distinction saves time in debugging, because it points attention to the right layer immediately.

Phase 2: Execution and Message Transport

During execution, nodes communicate through bounded channels associated with workflow edges:

- senders reserve queue capacity,
- packets are enqueued/dequeued at typed ports,
- queue pressure is observable via metadata records,
- backpressure is explicit through bounded capacity behavior.

This gives deterministic pressure points for debugging and policy decisions.

It is a small but important detail. Unbounded queues make it too easy to hide a design problem behind memory growth. Bounded queues force the system to say “slow down” in a way people can actually observe.

That is why the guide keeps coming back to bounded channels. They are one of the main reasons the runtime feels legible.

Phase 3: Terminalization and Summary

When execution reaches a terminal state (`completed`, `failed`, or `cancelled`), Pureflow emits:

- final metadata records (including terminal lifecycle and error records),
- `WorkflowRunSummary` with status, counts, and metadata path totals.

The summary is the machine-facing endpoint for automation and CI decisions.

That split is deliberate. The metadata stream is for forensics and live inspection. The summary is for the final answer. Most systems blur those two ideas together; Pureflow keeps them separate so each one stays simpler.

The result is that automation can stay fast while humans still have enough history to understand a run afterward.

Error Handling Model

Pureflow uses stable error taxonomies (`CDT-*`) and structured error payloads so consumers can branch behavior without string parsing.

The goal is not just nicer errors. The goal is errors that stay meaningful after the system grows.

Error surfaces are intentionally split:

Surface	When it appears	Why it matters
Validation errors	Before scheduling starts.	Keeps bad documents out of runtime entirely.
Runtime errors	During node execution or transport.	Points at a real execution failure.
Terminal summary error	At the end of `run --json`.	Gives automation one canonical decision object.

Retry/recovery policy remains a node/runtime concern and is represented through contract/core policy types rather than ad hoc CLI-only logic.

That keeps retry behavior where it belongs: with the runtime and the contract, not buried in a command handler.

Observability Touchpoints

Observability is first-class in the runtime model and includes:

This part makes runs easier to understand after the fact.

Record family	Tells you	Why it matters
`lifecycle`	State transitions across the run.	Shows the lifecycle of the execution itself.
`message`	Where packets moved.	Lets you trace flow without dumping payloads.
`queue_pressure`	Where queues were full or empty.	Makes backpressure visible.
`error`	How a failure was classified.	Supports policy, alerting, and debugging.
`external_effect`	Which side effects were observed.	Makes external behavior explicit and auditable.

That is part of why the runtime model is pleasant to work with. You are not left inferring state from a pile of ad hoc logs; the engine tells a consistent story about the run.

That consistency matters more than verbosity.

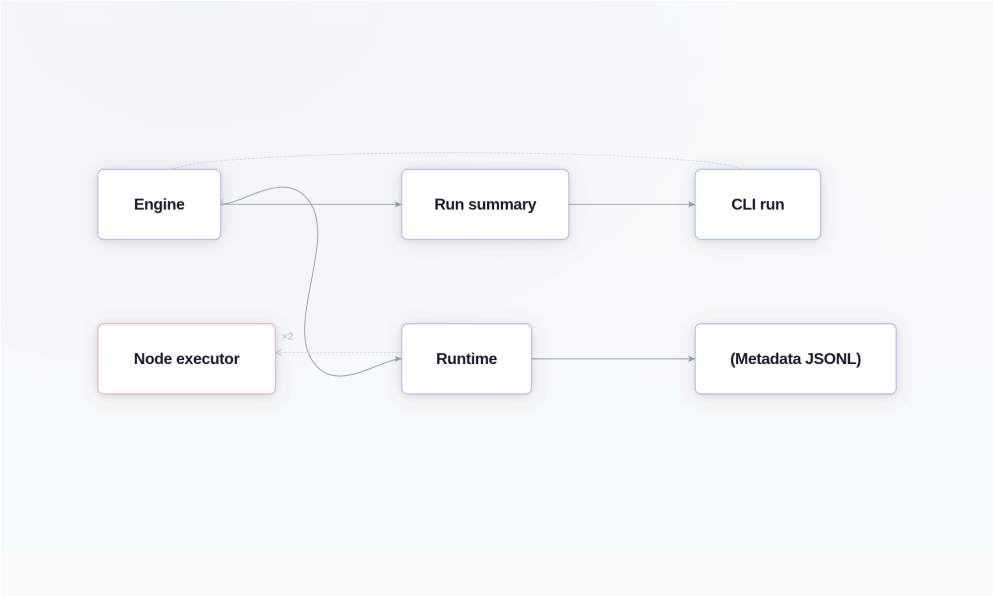

Figure 4: Observability flow diagram

Operational Notes

- Queue capacities in workflow edges are part of runtime behavior, not only structural metadata.
- Metadata JSONL is designed for tooling and dashboards; it is the canonical event stream for run forensics.
- The run summary is intentionally compact and should be preferred for fast pass/fail automation.

- Cancellation is represented as a shared Pureflow-owned signal, so a node can see the same request that the runtime supervisor sees.
- The runtime wrapper intentionally stays narrow. Most of the interesting behavior belongs to Pureflow itself, not to the async substrate.

Those two notes are the practical summary of this chapter.

Runtime Takeaways

The runtime model is not trying to be clever. It is trying to make the flow of work, the points of pressure, and the final outcome easy to follow.

That is why the chapter keeps returning to bounded channels, explicit metadata, and a narrow runtime boundary. If those pieces stay clear, the execution model stays teachable.

Package and Contract Boundaries

Intent

Define crate-level roles and the contract boundaries that keep workflow authoring, runtime execution, and integration adapters decoupled.

This chapter matters because crate boundaries are not just a packaging detail here. They are the line between "the thing Pureflow promises" and "the thing a particular backend happens to use today."

If the system starts to feel too abstract, this is the chapter to return to. It shows which parts are foundational, which parts are policy, and which parts are just adapters.

Crate Map by Responsibility

The map below is the short version of the crate split.

Think of it as a reading guide for the repository rather than a dependency chart.

Layer	Crates	What it keeps separate
Definition and validation	pureflow-types, pureflow-workflow-format, pureflow-workflow	Identifiers, file parsing, and graph shape.
Contract and policy	pureflow-contract, pureflow-core, pureflow-introspection	What a node promises, what it may do, and how we inspect it.

Layer	Crates	What it keeps separate
Execution and adapter	pureflow-engine, pureflow-runtime, pureflow-wasm	Orchestration, runtime substrate, and WASM boundaries.
Surface and test	pureflow-cli, pureflow-test-kit	User entrypoints and repeatable fixtures.

Public Contract Surfaces

Core contract surfaces are intentionally narrow and explicit:

Surface	What it describes	Why it stays separate
Workflow contract	Topology, ports, and edge capacities.	Keeps graph shape independent from node behavior.
Node contract	Input/output expectations and execution constraints.	Lets a node change without rewriting the workflow model.
Capability surface	What a node may do in a run environment.	Makes policy explicit instead of ad hoc.
Runtime trait surface	How executors plug into orchestration.	Keeps the execution seam narrow.
Metadata/error surface	Machine-facing events and terminal summaries.	Stabilizes automation and debugging.

The design goal is that each surface can evolve with additive compatibility, without requiring cross-cutting rewrites in unrelated crates.

That is the real reason for the split. If a new execution backend or adapter shows up later, the rest of the system should not have to relearn the workflow format, the contract model, or the metadata schema just because the runtime changed.

Put differently: each crate answers one question.

- What is the workflow?
- What is allowed?
- How do we execute it?
- How do we observe it?

When those questions stay separate, the code stays easier to read and harder to break accidentally.

The main architectural benefit of the split is that it keeps the system from turning every change into a debate about everything else.

Invariants

The following invariants should hold across releases:

These are the guardrails that keep the split useful.

- Structural graph validity is enforced before execution.
- Contract/capability mismatch blocks execution startup.
- Port schema incompatibility is detected before runtime scheduling.
- Metadata/error record families stay stable for downstream automation.
- Runtime adapters do not redefine core workflow semantics.

Those invariants are what make the architecture feel stable from a human point of view. A junior engineer should be able to change one layer without accidentally redesigning three others.

That stability is the reason the table exists at all.

Compatibility and Versioning Expectations

Workflow Format Compatibility

Area	Expectation	Reason
Workflow format	Additive when possible.	Keeps old documents readable.
Contract/capability	Deterministic and auditable semantics.	Preserves trust in the boundary.
Metadata	Stable record families and additive fields.	Avoids breaking scripts and dashboards.

Contract/Capability Compatibility

When you add a feature, the useful question is not only “does this compile?” It is “which contract surface changed, and who needs to know?”

That is the quickest way to avoid smearing policy across layers.

Boundary Diagram

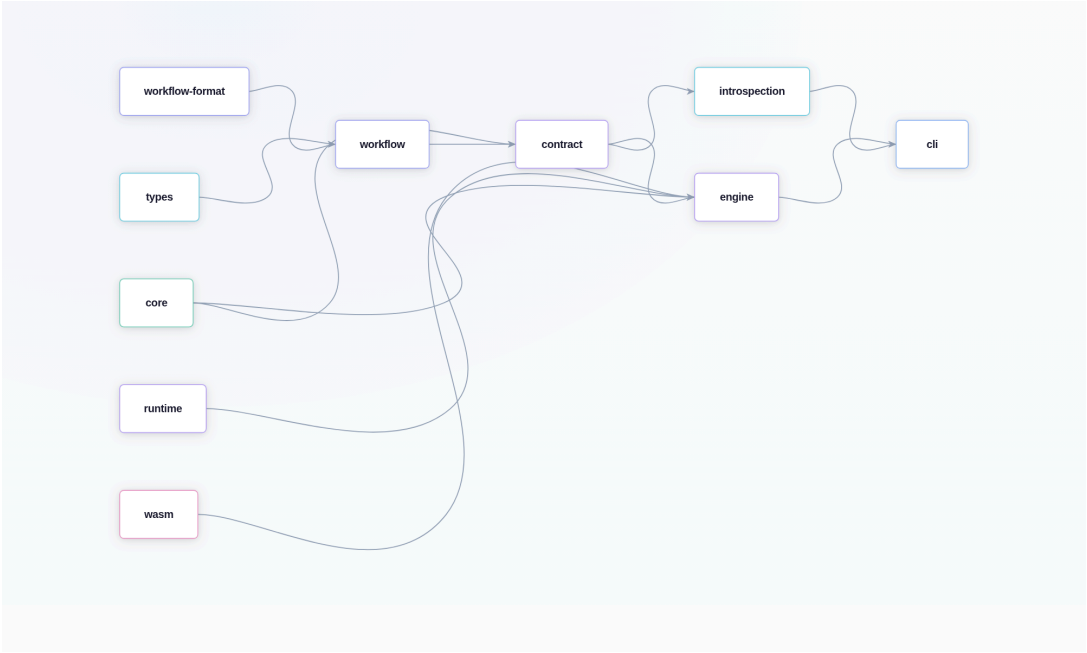

Figure 5: Package and contract boundary diagram

Review Checklist

When adding a feature, confirm:

- the owning crate is clear,
- boundary crossing is explicit (trait/contract/API),
- metadata/error impacts are documented,
- compatibility behavior is tested across old/new fixtures.

If you have to touch multiple crates to make one feature work, that is not automatically a problem. It just means the feature crosses a boundary that the guide is trying to keep explicit.

Crossing a boundary is fine. Losing sight of the boundary is what causes the mess.

Boundary Takeaways

The crate split is not there to make the tree look tidy. It exists so each layer can evolve on its own terms while still keeping the architecture comprehensible.

If a feature forces you to cross multiple crates, that is often a sign that the feature is real and the boundaries are doing useful work. The point is simply to cross them consciously.

Diagrams and Flows

Intent

Track canonical architecture diagrams and flow narratives used across the Pureflow architecture guide.

The diagrams support the prose; they do not replace it. The point of this guide is still to explain the architecture in words a person can sit with and read, with the figures acting as orientation rather than as the whole story.

This chapter is basically the map key for the rest of the book. It tells you what the diagrams are trying to prove and where to look if the layout changes.

Diagram Inventory

The inventory below keeps the figures grouped by the chapter they support.

That grouping matters more than it looks like it should. A diagram only helps if you can connect it back to the sentence that motivated it.

Chapter	Figures	Reader purpose
System Overview	Layered architecture; runtime sequence.	Shows how the major crates and runtime handoff fit together.
Runtime Model	Execution lifecycle; observability flow.	Shows how runs move, fail, and get summarized.
Package and Contract Boundaries	Crate boundary and ownership.	Shows where the design intentionally stops crossing boundaries.

Source of truth

Guide diagrams now live as checked Mermaid source under `docs/architecture-guide/figures/*.mmd` with rendered SVG assets under `docs/architecture-guide/figures/generated/*.svg` and PDF-safe PNG assets under `docs/architecture-guide/figures/generated-png/*.png`.

That split lets us review the source, inspect the generated artifact, and keep the PDF path stable.

Rendering Notes (Quarto + Mermaid)

Current rendering path for this guide:

Use this command when you want the whole book rebuilt from the checked sources.

```
nix develop . --command bash -lc 'cd docs/architecture-guide && quarto render --
to pdf'
```

PDF output path:

- `docs/architecture-guide/_output/Pureflow-Architecture-Guide.pdf`

For PDF stability:

- generate SVG assets with `frankenmermaid`,
- rasterize the SVGs to PNG for Quarto PDF output,
- keep the SVGs committed as the reviewable diagram artifact,
- keep labels concise to reduce node expansion in the source diagram.

Frankenmermaid Conventions

If/when `frankenmermaid` is used to generate diagrams, keep these boundaries:

These rules keep the generated artifacts useful instead of noisy.

- generated Mermaid source should be committed as reviewable input,
- generated PNGs should be committed as the PDF/HTML render artifact,
- chapter narrative remains authoritative over generated comments,
- long node labels should be wrapped or shortened before PDF render,
- diagrams should map to concrete crate/runtime concepts (no generic placeholders).

Canonical Flows to Preserve

These flows should remain represented as the guide evolves:

They are the recurring stories the diagrams need to keep telling, even after the surrounding prose changes.

1. Workflow ingestion to validation to execution.
2. Contract/capability/schema alignment before runtime start.
3. Runtime scheduling and packet movement through bounded queues.
4. Metadata emission and terminal summary production.
5. Native and WASM node execution boundaries.

Suggested Next Diagrams

Draft these as reviewable diagrams only after checking rendered symbols:

Use this list of diagrams when a chapter changes enough to deserve a visual update.

File	Chapter	What to verify
figures/system-overview-layered.mmd	System Overview	That the crate handoff story still matches the prose.
figures/runtime-model-lifecycle.mmd	Runtime Model	That lifecycle states still match current execution behavior.
figures/runtime-model-observability.mmd	Runtime Model	That the observability flow still matches emitted records.
figures/package-contracts-boundary.mmd	Package and Contract Boundaries	That crate ownership and boundary crossing are still accurate.

Review Workflow

When touching diagrams:

Use this as the quick workflow when a figure needs to change.

Diagram Takeaways

The diagrams are there to keep the reader oriented, not to replace the chapter text. If the prose and the figure disagree, fix the prose first and then bring the diagram back into line.

That keeps the guide readable even when the underlying implementation changes.

Step	What to do
1	Update the chapter narrative first.
2	Update Mermaid blocks to match current behavior.
3	Render PDF and check for page overflow.
4	Validate terminology against crate names and CLI output.